Eighteenth Implementation Iteration: Allocation Strategy Pattern

with Greedy Allocation Strategy and Service Delegation

Software Design / Iteration Report

Name: Najib Attar
Date: April 2026

1. Iteration Goal

The goal of this eighteenth implementation iteration is to improve the design flexibility of the prototype by introducing the Strategy Pattern for allocation algorithms.

In previous iterations, the `AllocationService` directly contained the logic for processing one-time and recurring requests. This worked correctly for the current prototype, but it made the service responsible for both coordination and allocation algorithm behavior.

In this iteration, the allocation algorithm is extracted into a strategy structure. A new allocation strategy interface is introduced, and a default `GreedyAllocationStrategy` is added to preserve the current one-by-one processing behavior. This creates a cleaner foundation for future allocation algorithms without changing the current rule evaluation flow or output behavior.

2. Changes from Iteration 17

Compared with the seventeenth iteration, the following changes were introduced:

- Added a new strategy interface: `IAllocationStrategy`
- Added a concrete strategy class: `GreedyAllocationStrategy`
- Moved the existing request-processing algorithm from `AllocationService` into the greedy strategy
- Refactored `AllocationService` so that it delegates processing to a strategy object
- Preserved the existing public behavior of `AllocationService`
- Preserved request history logging during evaluation, approval, and rejection
- Preserved CSV loading, allocation export, and request result export behavior

3. Scenario

The prototype starts by loading users, spaces, and requests from external CSV files through the `DataController`. After loading is completed, the main program sends each request to the `AllocationService`.

Instead of directly performing the allocation algorithm, the service now delegates request processing to the default `GreedyAllocationStrategy`. The greedy strategy evaluates each request using the existing rule engine facade and processes requests one by one in the same way as before.

If rule evaluation succeeds, the request is approved and the corresponding allocation or allocations are created. If rule evaluation fails, the request is rejected with the same rejection reason logic already used by the prototype. At the end of execution, the system exports:

- accepted allocations to `allocations.csv`
- full processed request outcomes to `request_results.csv`
- request lifecycle history together with each exported request result

This means the user-visible behavior remains the same, while the internal allocation design becomes more extensible.

4. Modules and Their Tasks

A. Allocation Strategy Module

- Define a common interface for allocation algorithms through `IAllocationStrategy`
- Allow different allocation approaches to be introduced later without redesigning the service layer
- Provide a clean location for algorithm-specific request processing logic

B. Greedy Allocation Module

- Implement the current one-by-one request processing behavior
- Evaluate requests using the existing `RuleEngineFacade`
- Approve requests and create allocations when rules pass
- Reject requests and preserve rejection reasons when rules fail

C. Allocation Service Module

- Continue owning the current allocation list
- Delegate allocation decisions to the selected strategy
- Preserve the existing public interface used by the main program

D. Request Management Module

- Continue supporting both one-time and recurring requests
- Preserve lifecycle history logging while requests are processed through the strategy

E. Data and Export Module

- Continue loading users, spaces, and requests from CSV files
- Continue exporting accepted allocations and processed request results
- Preserve request lifecycle history in the exported request results

5. Iteration Comparison

Iter.	Requests	Resources	Main Design Change
14	Same	Same	Structured user roles + role rule
15	Same	Same	Request priority model
16	Externalized requests	Class + Lab + Meet	DataController + CSV request loading
17	Same + history log	Class + Lab + Meet	Request lifecycle history logging + export
18	Same + strategy-based processing	Class + Lab + Meet	Allocation Strategy Pattern + Greedy strategy

6. Updated Class Diagram

Figure 1 shows the updated class diagram including the new allocation strategy interface, the greedy strategy implementation, and the delegation relationship from `AllocationService`.

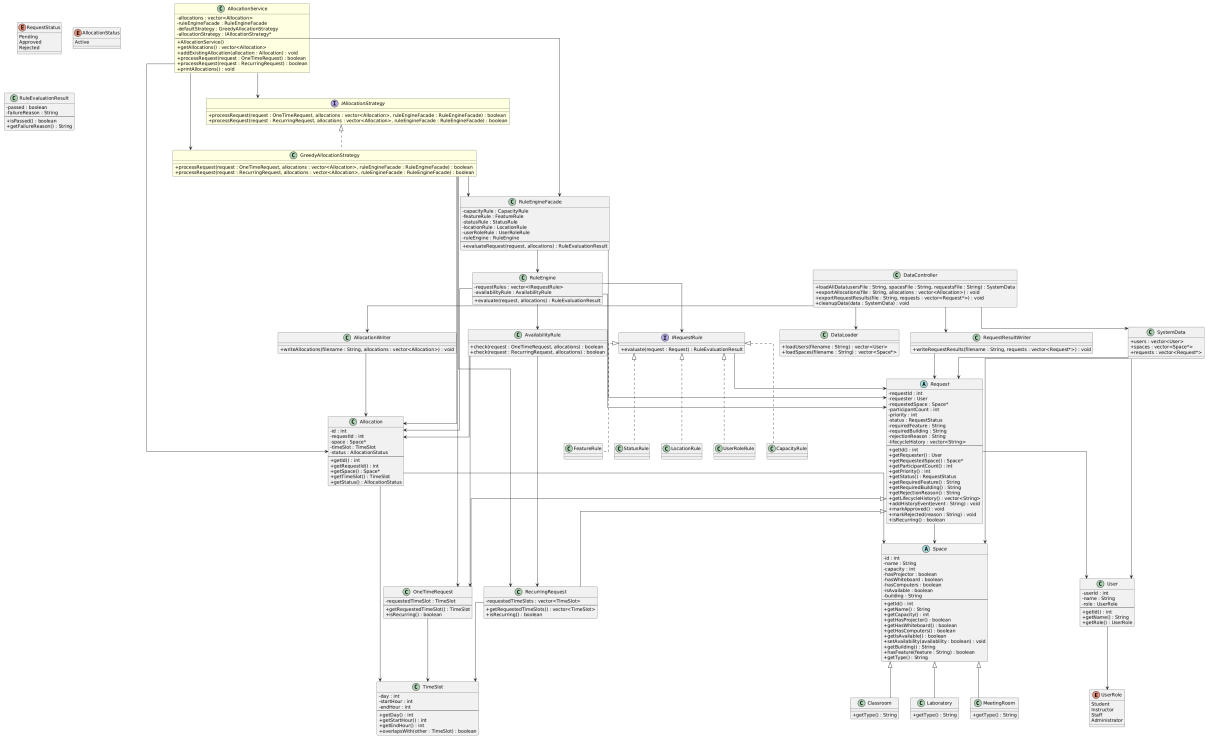

Figure 1: Updated Class Diagram with Allocation Strategy Pattern

7. Updated Workflow Diagram

Figure 2 shows the updated workflow. The overall processing sequence remains the same, but the allocation service now delegates request processing to the greedy strategy.

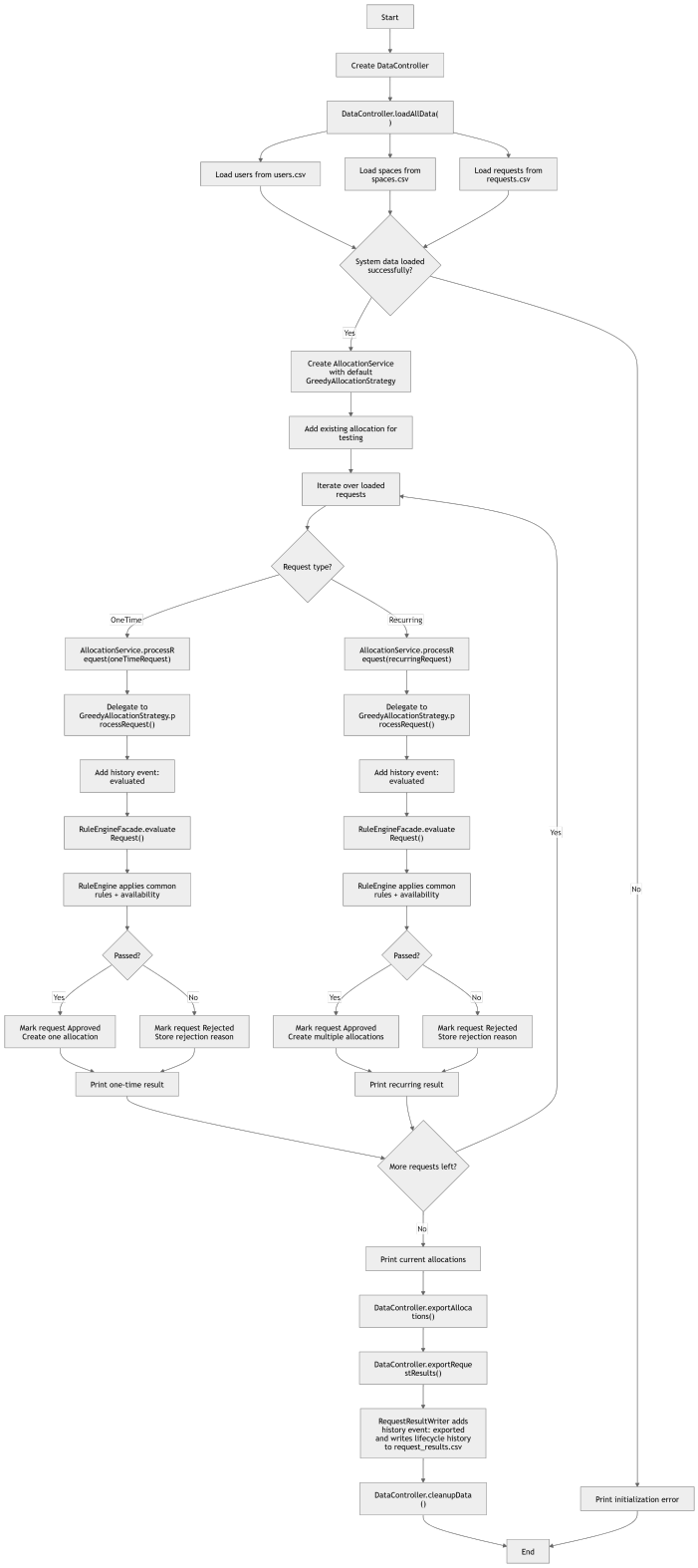

Figure 2: Workflow with Allocation Strategy Delegation

8. Prototype Implementation

The C++ prototype was extended by introducing an `IAllocationStrategy` interface and a concrete `GreedyAllocationStrategy`. The greedy strategy preserves the current behavior of the prototype by processing requests one at a time, evaluating them through the existing rule engine facade, and creating allocations only when all rules succeed.

The `AllocationService` was refactored so that it no longer contains the detailed allocation algorithm. Instead, it keeps ownership of the allocation list and delegates request processing to the selected strategy. In this iteration, the default strategy is the greedy strategy, so the observable behavior of the prototype remains unchanged.

This iteration improves separation of concerns because the service now coordinates processing while the strategy controls the algorithm details. It also provides a clean foundation for future iterations that may introduce alternative allocation algorithms such as priority-based processing, best-fit selection, or more advanced scheduling approaches.

9. Summary

This eighteenth implementation iteration improves the design of the prototype by introducing the Strategy Pattern for allocation algorithms. A new allocation strategy interface and a default greedy strategy were added, and the allocation service was refactored to delegate request processing to the strategy.

This iteration does not change the current functional behavior of the system. Requests are still processed correctly, allocations are still created in the same way, and CSV export behavior remains the same. However, the architecture is now better prepared for future extensions that require alternative allocation strategies.